Submission Worksheet

Submission Data

Course: IT202-008-S2026
Assignment: IT202 Milestone 1 - 2026
Student: Kurt C. (kc784)
Status: Submitted | **Worksheet Progress:** 100%
Potential Grade: 10.00/10.00 (100.00%)
Received Grade: 0.00/10.00 (0.00%)
Started: 4/9/2026 10:29:29 PM
Updated: 4/9/2026 10:29:29 PM
Grading Link: https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-milestone-1-2026/grading/kc784
View Link: https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-milestone-1-2026/view/kc784

Instructions

- Overview Link: https://youtu.be/lDtqdaLwcVg

1. Refer to Milestone1 of this doc: https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWVwfo/view
2. Ensure you read all instructions and objectives before starting.
3. Ensure you've gone through each lesson related to this Milestone
4. Switch to the Milestone1 branch
    1. `git checkout Milestone1` (ensure proper starting branch)
    2. `git pull origin Milestone1` (ensure history is up to date)
5. Fill out the below worksheet
    - Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
    - Ensure proper styling is applied to each page
    - Ensure there are no visible technical errors; only user-friendly messages are allowed
6. Once finished, click "Submit and Export"
7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
    1. `git add .`
    2. `git commit -m "adding PDF"`
    3. `git push origin Milestone1`
    4. On Github merge the pull request from `Milestone1` to `qa`
    5. On Github create a pull request from `qa` to `prod` and immediately merge. (This will trigger the `prod` deploy to make the heroku prod links work)
8. Upload the same PDF to Canvas
9. Sync Local
10. `git checkout qa`
11. `git pull origin qa`

Section #1: (2 pts.) Feature: User Will Be Able To Register A New Account

To Register A New Account

Progress: 100%

Task #1 (0.67 pts.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer
- Show the relevant demo of each validation layer
- Briefly explain how the validation steps work at each layer

Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

Progress: 100%

Details:

- Show the code related to the register page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)



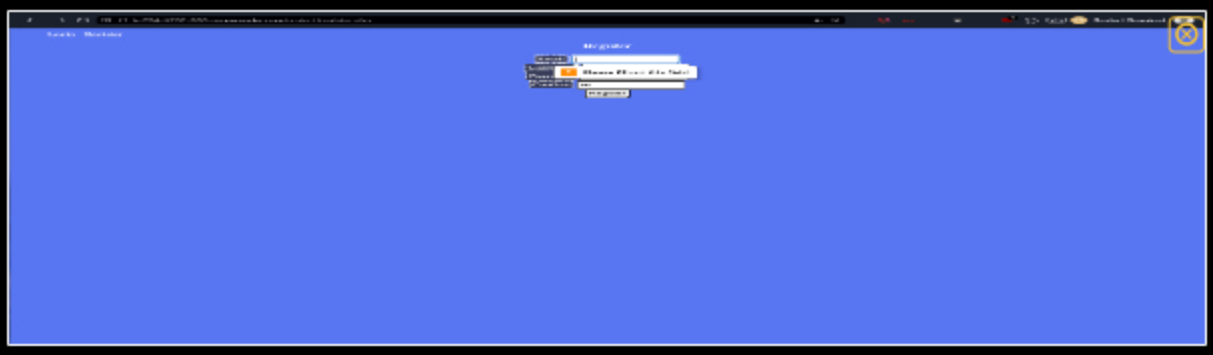
html validation



password length error



invalid email



empty email/username



empty password

Saved: 4/9/2026 7:00:28 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

I used the attributes: required, type="email", and minlength="8" in order to catch basic errors before submission. The sticky form logic uses se($_POST) on both the email and username

fields for repopulation if a submission fails so the user doesn't have to retype their information again.

Saved: 4/9/2026 7:00:28 PM

Task #2 (0.33 pts.) - JS Validation (validate() function)

Progress: 100%

Part 1:

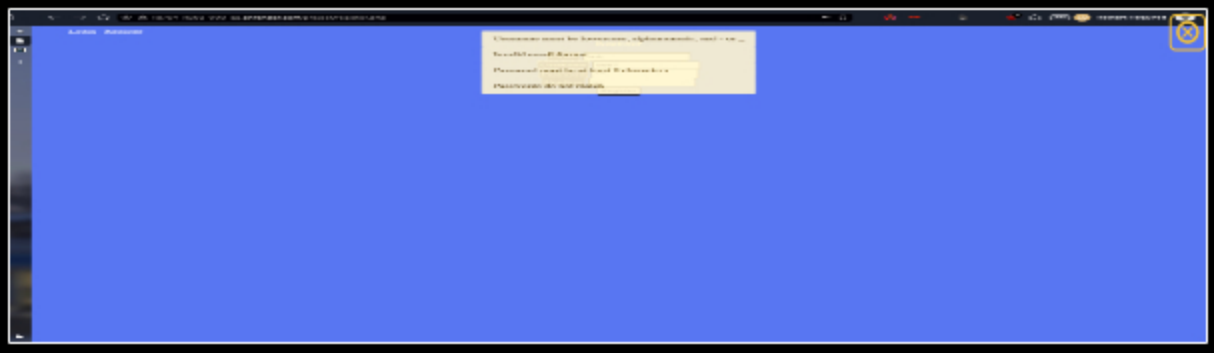
Progress: 100%

Details:

- Show the code related to the register page's validate() function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm](you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag



validate function



validation messages

Saved: 4/9/2026 7:09:54 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The validate() function runs before the submission of the form and checks each field with regex and length checks. flash() is used to display only user-firnedly warnings without submitting the form and returns false to stop submissions if a check fails.

Saved: 4/9/2026 7:09:54 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

Part 1:

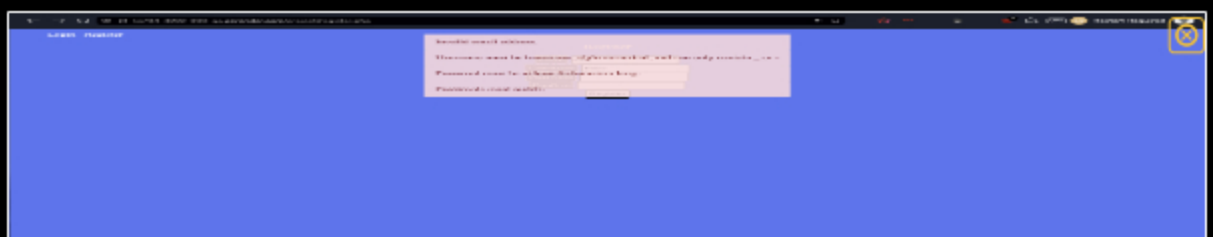
Progress: 100%

Details:

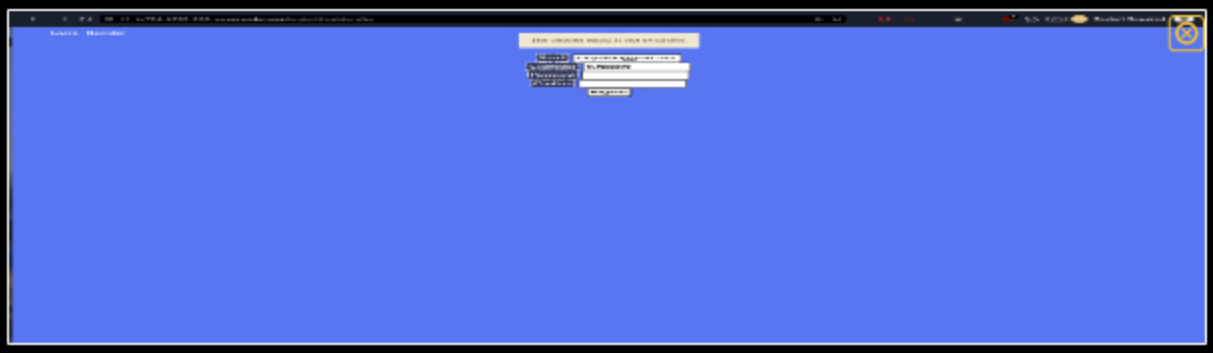
- Show the code related to the register page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password, password doesn't match confirm, username taken, email taken] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the validate() function or use the provided helper script in the instructions
- Include the outputs related to username/email already in use



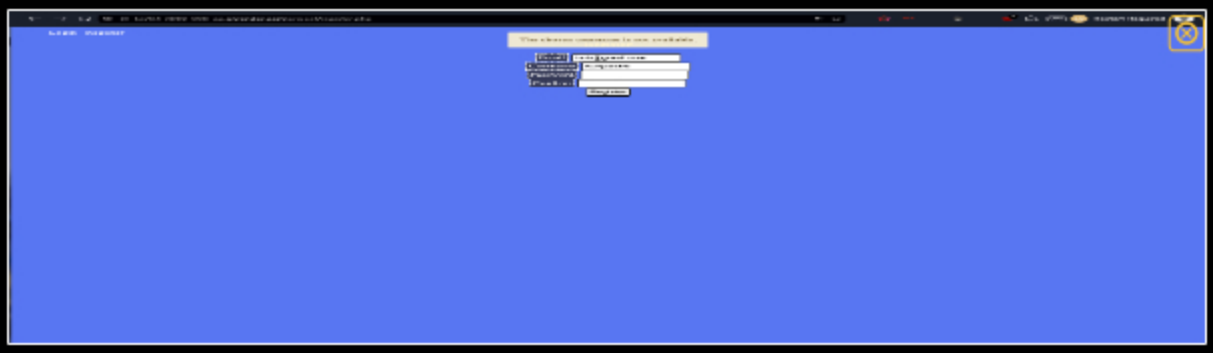
php validation code



validation errors



email unavailable



username not available

Saved: 4/9/2026 10:17:26 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The PHP validations checks each field using is_valid_email(), is_valid_username, and is_valid_password(). The $hasError flags takes those errors and displays the flash messages for the respective issue, including exiting email or usernames with users_check_duplicate().

Saved: 4/9/2026 10:17:26 PM

Task #2 (0.67 pts.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in `.php`, , zero credit if it does not)
- Include the render.com prod link to this file (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1
https://github.com/kurtjcastro/kc784-it202-008-2026/pull/15
URL: https://github.com/kurtjcastr

URL #2
https://kc784-it202-008-prod.onrender.com/project/register.php
URL: https://kc784-it202-008-prod.

URL #3
https://github.com/kurtjcastro/kc784-it202-[illegible]stone1/public_html/project/register.php
URL: https://github.com/kurtjcastr

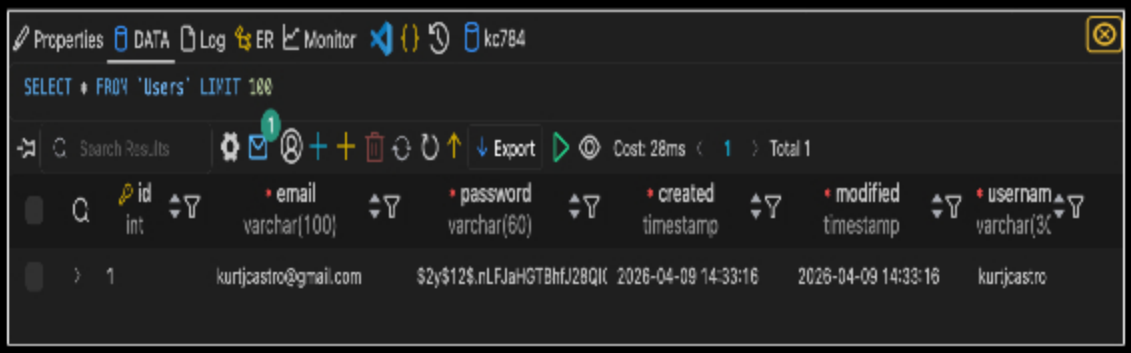
Saved: 4/9/2026 7:27:02 PM

Task #3 (0.67 pts.) - Database - Users table

Progress: 100%

Details:

- Add a screenshot of the database view from vs code showing a valid registered user (preferably a recent one during evidence capturing)



registered user

Saved: 4/9/2026 7:31:45 PM

Section #2: (2 pts.) Feature: User Will Be Able

To Login To Their Account

Progress: 100%

Task #1 (0.67 pts.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer (html, js, php)
- Show the relevant demo of each validation layer (html, js, php)
- Briefly explain how the validation steps work at each layer

Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

Progress: 100%

Details:

- Show the code related to the login page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)

```php
<?php
ob_start();// Temp fix to resolve output buffer issues that send the header() early that cause issues with the header("Location:...") bel
// kc784 | 4/9
// handles login, session creation, roles
require(__DIR__ . "/../../partials/nav.php");
?>
<h3>Login</h3>
<form onsubmit="return validate(this)" method="POST">
    <div>
        <label for="email">Email or Username</label>
        <input id="email" type="text" name="email" required
            value="<?php se($_POST, 'email', ''); ?>" />
    </div>
    <div>
        <label for="pw">Password</label>
        <input type="password" id="pw" name="password" required minlength="8" />
    </div>
    <input type="submit" value="Login" />
</form>
```

html validation



empty username



empty password



short password

Saved: 4/9/2026 7:37:07 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

Required is used on both username and password fields and minlength="8" is used to check the length of the password before it reaches the server. The sticky form logic goes through se($_POST, 'email', '') on login so that it remains populated after a failed submission to avoid retyping.

Saved: 4/9/2026 7:37:07 PM

Task #2 (0.33 pts.) - JS Validation (validate() function)

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the login page's validate() function
- Show examples of each validation message [username format, email format, password format] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag

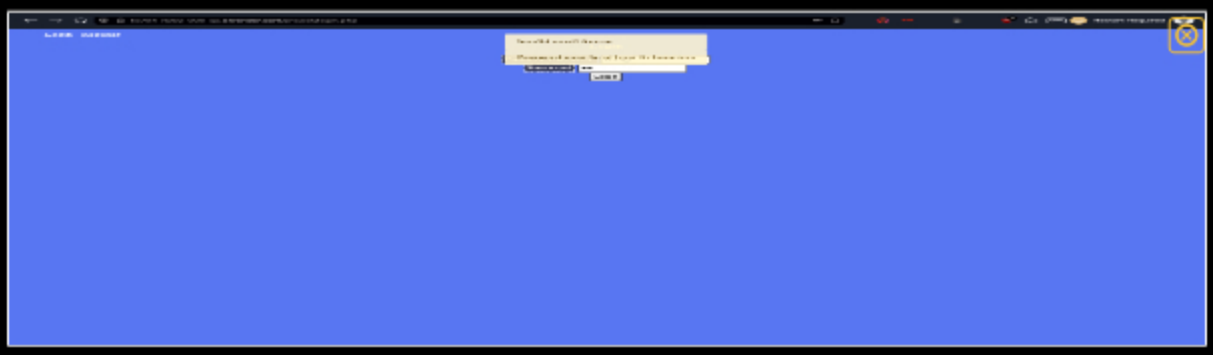


```
<script>
    function validate(form) {
        // TODO 1: implement JavaScript validation (you'll do this on your own towards the end of Milestone
        //ensure it returns false for an error and true for success

        // KC/B4 | 4/9
        // handles login, session creation, roles
        let login = form.email.value;
        let password = form.password.value;
        let isValid = true;

        if (login.includes("@")) {
            if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(login)) {
                flash("Invalid email format", "warning");
                isValid = false;
            }
        } else {
            if (!/^[a-z0-9_-]{3,20}$/.test(login)) {
                flash("Invalid username format", "warning");
                isValid = false;
            }
        } <-- #30-41 else
        if (password.length < 8) {
            flash("Password must be at least 8 characters", "warning");
            isValid = false;
        }
        return isValid;
    } <-- #21-47 function validate(form)
</script>
```

validate() function



invalid email format + short password



invalid username + short password

Saved: 4/9/2026 7:42:03 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

Validate() checks if the user input has @ to determine the validation as an email or username. If either the username or email is invalid or if the password is shorter than 8 chars, flash is used to issue a warning message and returns false to prevent submission.

Saved: 4/9/2026 7:42:03 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the login page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the validate() function or use the provided helper script in the instructions
- Include the outputs related to user doesn't exist and php-side password mismatch (the one that checks against the DB hash)



php validation



invalid email + short password



invalid email format



invalid username format



invalid login

Saved: 4/9/2026 10:19:39 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The validation checks the input by detecting the user field if it contains an @ to validate either as a username or email with is_valid_email() or is_valid_username(). If the login info does not match an existing user or if password fails with password_verify() a generic error message is thrown to avoid giving out too much information.

Saved: 4/9/2026 10:19:39 PM

Task #2 (0.67 pts.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in `.php` , , zero credit if it does not)
- Include direct link to the file on Render.com Prod (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1
https://github.com/kurtjcastro/kc784-it202-008-2026/tree/Milestone1/public_html/project/login.php

URL
https://github.com/kurtjcastr

URL #2
https://kc784-it202-008-prod.onrender.com/project/login.php

URL
https://kc784-it202-008-prod.

URL #3
https://github.com/kurtjcastro/kc784-it202-008-2026/pull/21

URL
https://github.com/kurtjcastr

Saved: 4/9/2026 7:54:45 PM

Task #3 (0.67 pts.) - User Session Evidence

Progress: 100%

Details:

```
- From the render.com logs (Logs tab), capture the session err
- It should show the id, username, email, and roles
```



output

Saved: 4/9/2026 8:00:26 PM

Section #3: (1 pt.) Feature: User Will Be Able To Logout

Progress: 100%

Task #1 (1 pt.) - Evidence

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the successful logout message (visit the proper file on Render.com qa after a manual deploy)
- Show the code snippet of the logout page



logout message

```
You, now | 2 authors (kurt and one other)
<?php
// kc784 | 4/9
// handles logout + message
// require functions.php to pull in flash()
require(__DIR__ . "/../../lib/functions.php");
reset_session(); // clear session data and start a new session
flash("You have been logged out","success");
header("Location: " . get_url("login.php")); // redirect back to login
```

logout code

Saved: 4/9/2026 10:20:34 PM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature

URL #1
https://github.com/kurtjcastro/kc784-it202-008-20261621

URL
https://github.com/kurtjcastro/kc

Saved: 4/9/2026 10:20:34 PM

Part 3:

Progress: 100%

Details:

- Briefly explain how the logout process works and how the user is officially logged off

Your Response:

When a user logs out, the reset_session() destroys the current session and creates a new one. This clears all previous user data and their login roles and then a flash message confirms logout success and the user is redirected to the login page. The back button is also prevented from granting access to the previous logged in state.

Saved: 4/9/2026 10:20:34 PM

Section #4: (2 pts.) Feature: Security Rules

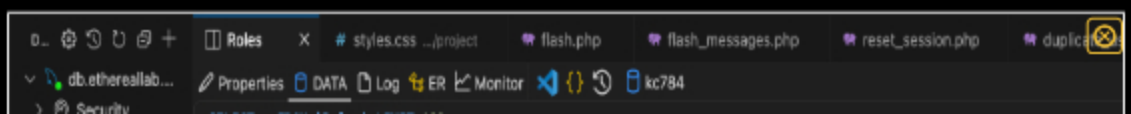
Progress: 100%

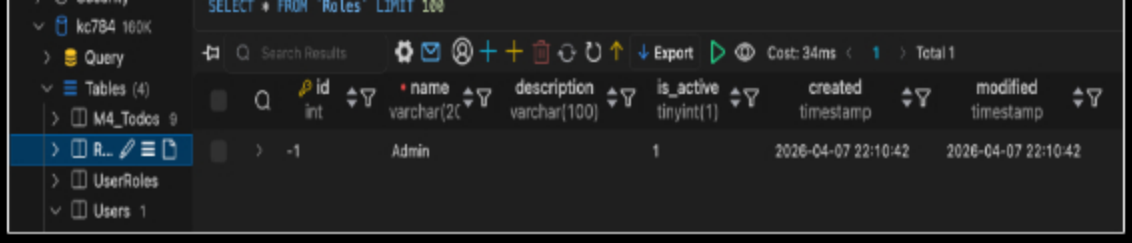
Task #1 (1 pt.) - Database Tables

Progress: 100%

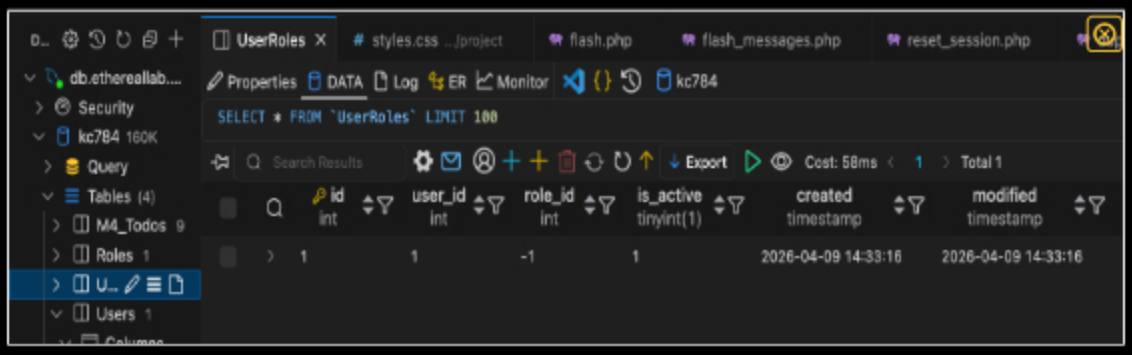
Details:

```
- From the vs code mysql tool, show both the `Roles` and `User
```





roles



userroles

Saved: 4/9/2026 8:55:38 PM

Task #2 (1 pt.) - Demo Security Rules

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the message related to needing to be logged in (i.e., try to
- Show the message related to not having the proper permission/role
- Include a snippet of the login check function
- Include a snippet of the role check function



must be logged in



no permission

```php
// kc784 | 4/9
// checks logged in state
function is_logged_in($redirect = false, $destination = "login.php")
{
    $isLoggedIn = isset($_SESSION["user"]);
    if ($redirect && !$isLoggedIn) {
        //if this triggers, the calling script won't receive a reply since die()/exit() terminates it
        flash("You must be logged in to view this page", "warning");
        $path = get_url($destination);

        die(header("Location: $path"));
    } <- #14-20 if ($redirect && !$isLoggedIn)
    return $isLoggedIn;
} <- #12-22 function is_logged_in($redirect = false, $destination = "logi...
```

login check function

```php
// kc784 | 4/9
// checks role
function has_role($role)
{
    if (is_logged_in() && isset($_SESSION["user"]["roles"])) {
        foreach ($_SESSION["user"]["roles"] as $r) {
            if ($r["name"] === $role) {
                return true;
            }
        } <- #28-32 foreach ($_SESSION["user"]["roles"] as $r)
    } <- #27-33 if (is_logged_in() && isset($_SESSION["user"]["roles"]))
    return false;
} <- #26-35 function has_role($role)
```

role check function

Saved: 4/9/2026 10:26:10 PM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature

URL #1
https://github.com/kurtjcastro/kc784-it202-008-20[illegible]21

URL
https://github.com/kurtjcastro/kc

Saved: 4/9/2026 10:26:10 PM

Part 3:

Progress: 100%

Details:

- Briefly explain how the login check code works
- Briefly explain how the role check code works

Your Response:

is_logged_in() checks if the user key exists within the session and if it does, the user is considered as logged in. When $redirect is true on protected pages, a flash message is sent and redirects the user to the login page to avoid showing them role-sensitive information.

has_role() confirms if the user is logged in and loops through the roles stored within the session and matches the name to the role. If it finds a match, true is returned and if not, then access is denied to the protected page.

Saved: 4/9/2026 10:26:10 PM

Section #5: (2 pts.) Feature: User Profile

Progress: 100%

Task #1 (1 pt.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer (html, js, php)
- Show the relevant demo of each validation layer (html, js, php)
- Briefly explain how the validation steps work at each layer

Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

Progress: 100%

Details:

- Show the code related to the profile page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or

few screenshots) (visit the proper file on Render.com qa after a manual deploy)



html validation



invalid email format



invalid username



short password + password mismatch

Saved: 4/9/2026 10:27:10 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

Profile.php uses type="email" in the email field for basic format validation before the form submission and the sticky form logic prefills the email and username with se($email) and se($username) to pull from the session so the fields remain populated with the user input even after a failed submission

Saved: 4/9/2026 10:27:10 PM

Task #2 (0.33 pts.) - JS Validation (validate() function)

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the profile page's validate() function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag



validate() function



invalid username format, email format, password length, password mismatch

Saved: 4/9/2026 9:33:33 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The validate() function checks the username and email format and the length of the password if a new password is being entered, as well as checking for a match when the user retypes the new password. The function return false and flashes a generic message if any check failed in order to prevent the form from being submitted.

Saved: 4/9/2026 9:33:33 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the profile page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password, password doesn't match confirm, username taken, email taken] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the validate() function or use the provided helper script in the instructions

```php
// kc784 | 4/9
// handle email/username update
if (isset($_POST["email"], $_POST["username"])) {
    $new_email = se($_POST, "email", null, false);
    $new_username = se($_POST, "username", null, false);
    $hasError = false;
    // validate format
    if (empty($new_email)) {
        //echo "Email must not be empty<br>";
        flash("Email must not be empty.", "danger");
        $hasError = true;
    } <= #21-25 if (empty($new_email))
    // Sanitize and validate email
    $new_email = sanitize_email($new_email);
    if (!is_valid_email($new_email)) {
        //echo "Invalid email address<br>";
```

```
    flash("Invalid email address.", "danger");
    $hasError = true;
} -- #20-32 if (!is_valid_email($new_email))
if (!is_valid_username($new_username)) {
    flash("Username must be lowercase, alphanumerical, and can only contain _ or -", "danger");
    $hasError = true;
}
```

php validation



password validation



invalid username format, email format, password length, password mismatch



existing username

Saved: 4/9/2026 10:29:29 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The PHP validation checks username and email formating with is_valid_email() and is_valid_username() functions and also checks password changes by verifying if the length is at least 8 characters and if the new and confirm passwords match through is_valid_confirm(). Any duplicate emails or usernames are seen by the database and sends a user-friendly message to the user that the name is not available.

Saved: 4/9/2026 10:29:29 PM

Task #2 (1 pt.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in `.php` , , zero credit if it does not)
- Include direct link to the file on Render.com Prod (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1
https://github.com/kurtjcastro/kc784-it202-008-2026/pull/21
URL: https://github.com/kurtjcastr

URL #2
https://kc784-it202-008-prod.onrender.com/project/profile.php
URL: https://kc784-it202-008-prod.

URL #3
https://github.com/kurtjcastro/kc784-it202-008-2026/Milestone1/public_html/project/profile.php
URL: https://github.com/kurtjcastr

Saved: 4/9/2026 9:48:39 PM

Section #6: (1 pt.) Misc

Progress: 100%

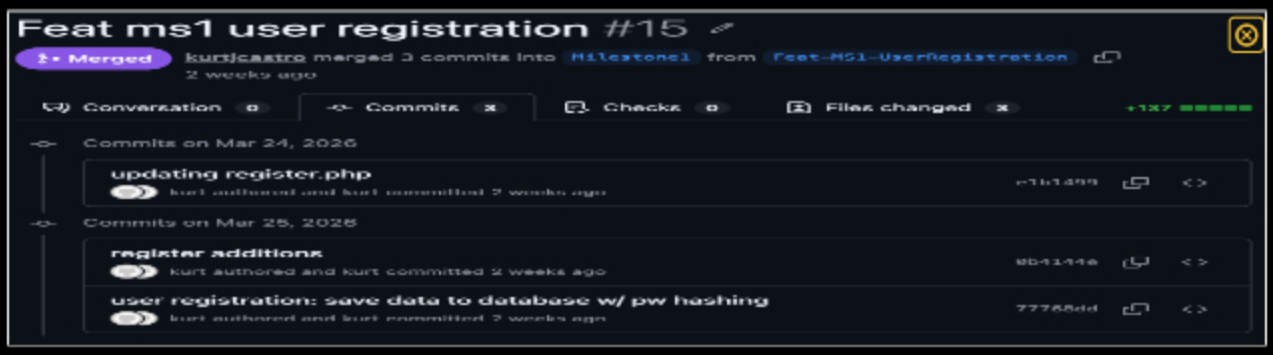
Task #1 (0.33 pts.) - Github Details

Progress: 100%

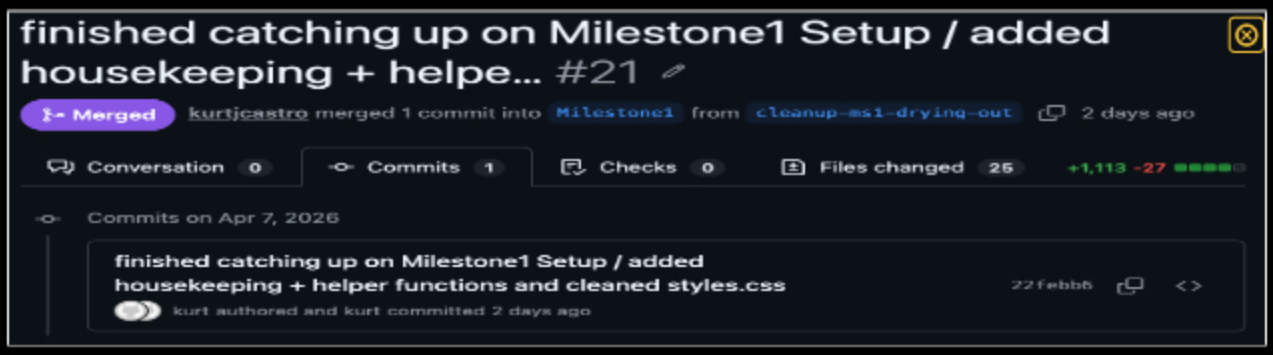
Part 1:

Progress: 100%

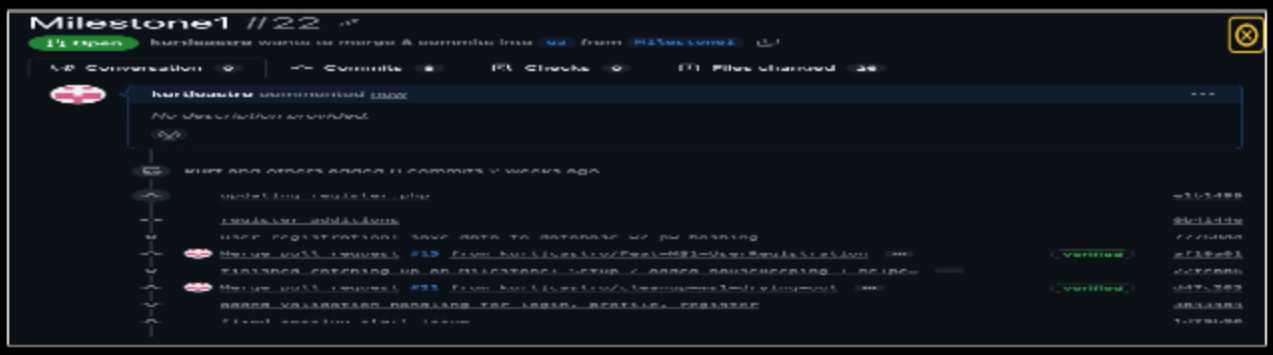
Details:
From the Commits tab of the Pull Request screenshot the commit history (it may not be possible to capture all, just grab a reasonable chunk)



registration



followed lessons from login to milestone1 cleanup to catch up but forgot to commit from new branches. all additions from lessons



all commits i have (missed a few due to forgetting to make new branches for lessons)

Saved: 4/9/2026 9:57:43 PM

Part 2:

Progress: 100%

Details:
Include the link to the Pull Request (should end in `/pull/#`) (Milestone1 into qa)

URL #1
https://github.com/kurtjcastro/kc784-it202-008-2026/pull/21

URL
https://github.com/kurtjcastro/kc

URL #2
https://github.com/kurtjcastro/kc784-it202-008-2026/pull/22

URL
https://github.com/kurtjcastro/kc

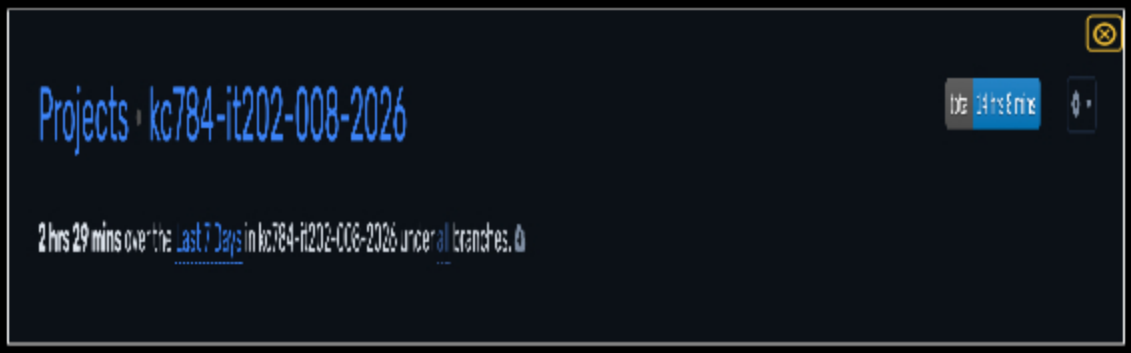
Saved: 4/9/2026 9:57:43 PM

Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click `Projects` and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



overall time



file time

Saved: 4/9/2026 9:58:49 PM

Task #3 (0.33 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

I learned how to set up an authentication system in PHP with registration, session management, login, and roles/permissions. I also learned how to validate user input with PHP and HTML validations.

Saved: 4/9/2026 10:11:27 PM

Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part was setting up the HTML forms because the markup and built-in validation such as required and type="email" made it simple and straightforward.

Saved: 4/9/2026 10:12:25 PM

Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The most difficult part of the assignment was understanding how to manage sessions and roles to make sure that roles were only stored and assigned correctly after a successful login. This was crucial in ensuring only validated users could access protected pages and unauthorized users would be redirected to the login page and given a generic error message.

Saved: 4/9/2026 10:14:33 PM